技术部第五次技术分享会

枚举、泛型与常用标准库

姚雨森　技术部分管主席

用整数表示状态，需要记住编号的含义

一个订单可能处于待付款、已付款、已发货或已取消状态。

```
int status = 0;

if (status == 1) {
    System.out.println("Ready to ship");
}

status = 7;
```

- 阅读代码时，需要另外查找：0 和 1 分别代表什么？
- int 允许许多整数，但业务只需要四种状态。
- 即使 7 没有业务含义，赋值语句仍能通过类型检查。

换成字符串更直观，但仍然可能写错

```
String status = "PENDING";
status = "PAID";

status = "PAIID";
status = "paid";
```

- 名称比编号容易理解，但任何字符串都可以赋给 status。
- 拼写错误、大小写不一致，都可能让后面的判断失效。
- 我们希望同时表达每个选项的含义和允许使用的选项。

枚举：为有限的一组选择定义类型

订单状态只需要以下四个选项：

含义	枚举常量名称
待付款	PENDING
已付款	PAID
已发货	SHIPPED
已取消	CANCELLED

枚举（enum）将这些选项列在一起，定义一个专门的类型。

星期、方向、任务状态等有限选项，也可以用枚举表达。

使用 enum 定义订单状态类型

将下面的定义放在 OrderStatus.java 中：

```
public enum OrderStatus {
    PENDING,
    PAID,
    SHIPPED,
    CANCELLED
}
```

- enum 用于声明枚举；OrderStatus 是新类型的名称。
- 花括号中列出枚举常量，使用逗号分隔。
- 类型名称通常使用大驼峰；常量通常使用大写字母和下划线。
- 使用常量时写成类型名. 常量名，例如 OrderStatus.PAID。

像其他类型一样声明变量和赋值

以下语句写在 Main.java 的 main 方法中，两个源文件放在同一目录。

```
OrderStatus status = OrderStatus.PENDING;
System.out.println(status); // PENDING

status = OrderStatus.PAID;
System.out.println(status); // PAID
```

- 左边的 OrderStatus 指定变量类型。
- 右边的 OrderStatus.PENDING 是该类型的一个常量。
- 变量可以重新赋值；这里是改为保存另一个枚举常量。

类型检查帮助我们排除无效的赋值

下面的错误写法用于说明类型限制，不能通过编译：

```
OrderStatus status = OrderStatus.PENDING;

status = 1;                    // Wrong type
status = "PAID";               // Wrong type
status = OrderStatus.PAIID;    // No such constant
```

- 枚举常量有自己的类型；字符串 "PAID" 并不是它。
- 拼错常量名称时，编译器就能指出问题。
- 另一种枚举类型的常量，即使同名，也不能赋给这个变量。

补充：枚举变量也可以为 null，表示没有引用枚举常量；本例始终给它赋明确的状态。

使用 == 判断是否为某个枚举常量

```
OrderStatus status = OrderStatus.PAID;

if (status == OrderStatus.PAID) {
    System.out.println("Ready to ship");
}

if (status != OrderStatus.CANCELLED) {
    System.out.println("Order is not cancelled");
}
```

- == 判断是否为同一个枚举常量；!= 判断是否不同。
- 直接比较枚举值，不需要先转换成编号或字符串。
- 字符串内容比较仍使用 equals，不要把这里的规则照搬过去。

枚举作为参数：明确方法接收哪一种状态

假设本例规定：只有待付款的订单可以取消。

在 Main 类中定义方法：

```
static boolean canCancel(OrderStatus status) {
    return status == OrderStatus.PENDING;
}
```

在 main 方法中调用：

```
boolean first = canCancel(OrderStatus.PENDING); // true
boolean second = canCancel(OrderStatus.SHIPPED); // false
```

参数的类型说明需要什么数据；方法体决定这组数据对应的业务规则。

商品价格需要满足非负的要求

回顾：通过 setter 修改商品的价格。本例价格用整数表示，单位为分。

```
public class Product {
    private int price;

    public void setPrice(int price) {
        this.price = price;
    }
}
```

- 字段设为 `private`，类外通过 `setPrice` 修改它。
- 目前调用 `setPrice(-10)`，仍然会把价格改为负数。
- 我们希望拒绝非法价格，并让调用方知道操作失败。

使用 throw 抛出异常，拒绝非法参数

将 Product 中的 setter 改为：

```
public void setPrice(int price) {
    if (price < 0) {
        throw new IllegalArgumentException(
            "Price must be non-negative");
    }
    this.price = price;
}
```

- IllegalArgumentException 表示参数不符合方法的要求。
- new 创建异常对象；字符串说明失败的原因。
- throw 抛出异常，本次调用不能正常完成。
- 检查在赋值之前：负数不会写入字段，0 则是合法价格。

异常类型帮助我们识别失败原因

异常类型	典型原因
`IllegalArgumentException`	参数不符合要求
`NumberFormatException`	文本不能转换为要求的数值
`NullPointerException`	对 `null` 进行需要对象的操作
`IndexOutOfBoundsException`	索引超出允许范围

- 参数校验失败时，可以主动抛出合适的异常。
- 调用库方法或执行其他操作时，也可能出现异常。
- 类型说明问题的类别，异常消息提供本次失败的具体信息。

异常没有被处理，调用方也不能照常继续

在 main 方法中调用修改后的 setter：

```
Product product = new Product();
product.setPrice(1000);
product.setPrice(-10);
System.out.println("Price updated");
```

- 第一次设置成功；第二次在 setter 中抛出异常。
- 异常传播到 main，最后的输出语句不会执行。
- 始终没有匹配的处理时，当前线程终止；本例程序随之结束。

控制台会给出异常类型、消息和调用堆栈；可以从中找到自己代码的文件名与行号。

在调用方使用 try-catch 处理设置失败

以下代码写在 main 方法中：

```
Product product = new Product();
product.setPrice(1000);
try {
    product.setPrice(-10);
    System.out.println("Price updated");
} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage());
}
System.out.println("Continue");
```

- try 中放置可能失败的操作。
- catch 捕获匹配的异常，变量 e 接收异常对象。
- e.getMessage() 取得异常消息，此处用它给出失败提示。

对照两条路径：正常执行与异常处理

将上一页的 `-10` 分别改为 `2000` 和 `-10`：

传入 2000：正常完成

1. 设置价格成功。
2. 输出 `Price updated`。
3. 跳过 `catch`。
4. 输出 `Continue`。

传入 -10：进入异常处理

1. setter 抛出异常。
2. 跳过 `try` 中剩余语句。
3. 进入匹配的 `catch`，输出原因。
4. 输出 `Continue`。

本例处理后，继续执行整个 `try-catch` 后面的语句，不会回到出错位置接着执行。

setter 检查规则，调用方处理失败

- **setter：**检查价格，拒绝负数，并抛出异常报告失败。
- **调用方：**根据场景提示用户、重新输入，或取消这次操作。
- **自己的代码写错时：**修正非法参数的来源，避免反复触发同一问题。

捕获异常不会自动修复数据，也不会自动重试

本例价格仍为原来的 `1000`，是因为 setter 先检查、后赋值。异常机制本身不会撤销此前已经完成的修改。

不要写空的 `catch` 来隐藏失败；在能作出合理处理的位置捕获异常。

补充：有些异常要求捕获或显式声明

- Java 对部分异常有编译要求：调用方必须捕获，或在方法上用 `throws` 声明可能向外传播。
- 本例的 `IllegalArgumentException` 没有这项强制要求，也可以使用 `try-catch` 处理。

区分两个关键字

`throw`：实际抛出异常。

`throws`：写在方法声明中，说明可能向外传播的异常，不代表已经处理了异常。

课后了解：受检异常与非受检异常的分类，以及 `throws` 的使用。

回顾：列表的尖括号指定元素类型

第四次分享中，我们已经使用过泛型：

```
// import java.util.ArrayList;
ArrayList<String> names = new ArrayList<>();
names.add("Alice");
String name = names.get(0);

ArrayList<Integer> scores = new ArrayList<>();
scores.add(90);
int score = scores.get(0);
```

- 都是 ArrayList，但一个保存字符串，另一个保存整数。
- 尖括号中的类型，让编译器检查添加的数据和读取结果的类型。
- **泛型（generics）**让同一份定义可以适用于不同类型。

定义 Box<T>：用类型参数表示保存的类型

先在 Box.java 中定义保存和读取操作：

```
public class Box<T> {
    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

- <T> 声明类型参数；T 用于字段、构造参数和返回类型。
- 定义时先用 T 表示类型，使用时再指定具体类型。

使用 Box<String>：保存和读取字符串

以下语句写在 main 中，Box.java 与入口文件放在同一目录：

```
Box<String> box = new Box<String>("Hello");
String text = box.get();
System.out.println(text); // Hello
```

- Box<String> 将这里的类型参数指定为 String。
- 编译器据此检查构造时传入的值，并确定 get( ) 的返回类型。
- 读取结果可以直接赋给 String 变量，不需要手工类型转换。

可以按“这里的 T 对应 String”理解类型检查，不必为字符串另外定义一个盒子类。

使用 Box<Product>：同一份定义保存商品

继续使用前面的 Product 类：

```
Product product = new Product();
product.setPrice(1000);

Box<Product> box = new Box<Product>(product);
Product saved = box.get();
saved.setPrice(2000);
```

- Box 的源码保持一致，这次指定的类型是 Product。
- 构造时传入商品，读取时得到 Product 类型的结果。
- 自己定义的类，也可以作为泛型的类型实参。

为 Box<T> 增加交换内容的操作

将下面的方法加入 Box<T> 类中，与 get() 并列：

```
public void swapWith(Box<T> other) {
    T temp = this.value;
    this.value = other.value;
    other.value = temp;
}
```

- other 是另一个保存同种类型数据的盒子。
- 用 T 类型的临时变量暂存当前值，再交换两个字段的内容。
- 方法定义在 Box 类内部，可以访问另一个盒子的私有字段。

交换过程不依赖具体数据类型；本例假设传入的 other 非 null。

调用 swapWith：观察交换前后的内容

以下语句写在 main 中：

```
Box<String> first = new Box<>("Apple");
Box<String> second = new Box<>("Banana");
System.out.println(first.get());  // Apple
System.out.println(second.get()); // Banana

first.swapWith(second);

System.out.println(first.get());  // Banana
System.out.println(second.get()); // Apple
```

- this 对应 first，参数 other 接收 second。
- 交换的是两个盒子保存的值；变量仍然指向各自原来的盒子。

同一段交换代码可以用于不同类型

对整数盒子，仍然使用相同的 swapWith 方法：

```
Box<Integer> first = new Box<>(10);
Box<Integer> second = new Box<>(20);
first.swapWith(second);
System.out.println(first.get());  // 20
System.out.println(second.get()); // 10
```

- 字符串、整数和商品盒子，都可以复用这段交换逻辑。
- 对 Box<Integer>，参数 Box<T> 对应 Box<Integer>。
- 类型仍需匹配：整数盒子不能与字符串盒子交换。

泛型既能描述保存的数据，也能描述与具体类型无关的操作。

泛型让类型不匹配在编译时暴露

已有一个保存字符串的盒子：

```
Box<String> box = new Box<String>("Hello");
String text = box.get(); // OK
```

下面两行分别是错误示例，不能通过编译：

```
Box<String> wrong = new Box<String>(123);
Integer number = box.get();
```

- 第一行需要字符串，却传入了整数。
- 第二行读取的是字符串，却试图赋给 Integer 变量。
- 编译器根据类型参数检查两端，帮助我们提前发现类型不匹配。

类型实参使用引用类型：Integer 而非 int

保存整数时，写成 Box<Integer>：

```
Box<Integer> box = new Box<Integer>(42);
int number = box.get();
System.out.println(number); // 42
```

- 泛型的类型实参不能直接使用 int、double 等基本类型。
- 对应的包装类包括 Integer、Double 和 Boolean。
- 本例传入 42 时自动装箱，读取并赋给 int 时自动拆箱。

与第四次分享中的 ArrayList<Integer> 使用相同的规则；Box<int> 是错误写法。

T 是参数名，尖括号中也可以有多个参数

回顾已经使用过的键值映射：

```
// import java.util.HashMap;
HashMap<String, Integer> stock = new HashMap<>();
stock.put("P1001", 8);
Integer count = stock.get("P1001");
```

- T 是常用的类型参数名，不是固定关键字；声明和使用应保持一致。
- 类型可以有多个参数，用逗号分隔，例如 HashMap<K, V>。
- 这里 K 对应键的类型 String，V 对应值的类型 Integer。

Box<T> 只有一个类型参数；需要几个参数，由类型定义决定。

右侧的 <> 可以省略重复的类型书写

显式写出构造时使用的类型：

```
Box<String> box = new Box<String>("Hello");
```

可以简写为：

```
Box<String> box = new Box<>("Hello");
```

- <> 称为菱形语法，编译器根据上下文推断类型实参。
- 简写后的类型检查与前面的写法一致。
- new Box<>(...) 中仍保留尖括号，不要写成省略泛型的原始类型。

泛型的重点：定义时保留类型参数，使用时指定类型，并由编译器检查。